WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

OBS仓颉版客户端SDK实现(OBS-client-cj)



主讲人：许飞锋

普元信息技术股份有限公司

个人简介



- ✓ 许飞锋
- ✓ 入职普元5年、微服务产品研发部门担任研发工程师
- ✓ 拥有5年Java开发经验 、接触仓颉语言两年
- ✓ 拥有多年的java微服务开发经验，对相关组件研究较深入

WORKSHOP

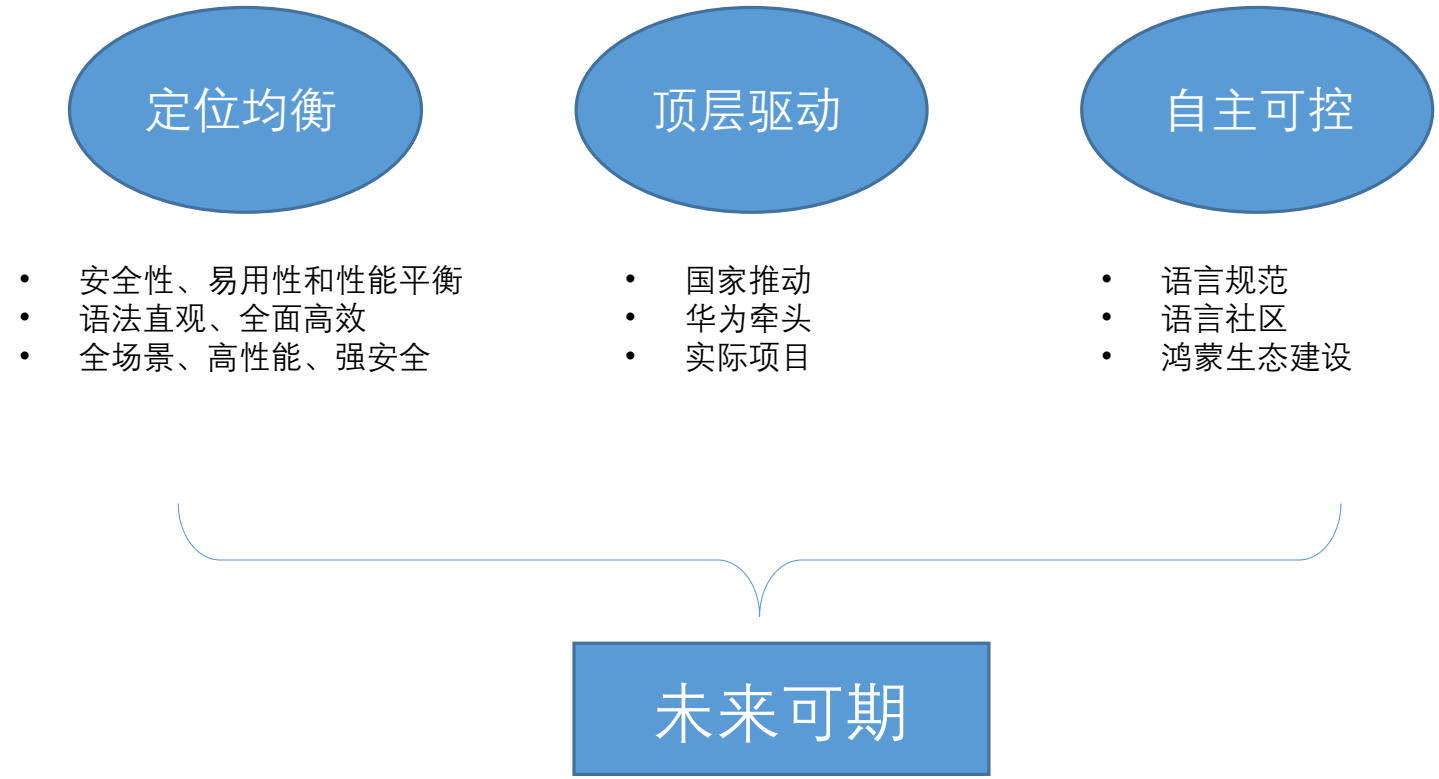
经历和感受

2023-5（首次接触）	使用仓颉语言实现ESB引擎监控管理模块
2023-10	使用仓颉语言实现OBS客户端组件SDK
2024-8	使用仓颉语言开发zookeeper客户端

容易入门	文档交齐全、示例代码简单直接、语法易懂
社区活跃	问题互动积极、日常开发培训、问题回答快速
应用开发简便	三方组件丰富、应用开发框架-神农框架
性能相对较好	底层语言编译执行、应用层接口实际测试

WORKSHOP

仓颉语言定位和看法



WORKSHOP

WORKSHOP 研讨/分享/感悟

库开发

01 背景情况

02 库介绍

03 特性分享

04 未来展望

仓颉 Cangjie

背景情况-库描述及目标

对象存储服务软件开发工具包(OBS-client-cj)是对OBS服务提供的REST API进行的封装，以简化用户的开发工作。用户直接调用OBS SDK提供的接口函数即可实现使用OBS服务业务能力的目的。

库组件目标：

- OBS仓颉版客户端SDK
- 实现OBS在Java中现有特性
- 性能上要不能低于Java同版本

Java版组件信息：

V:3.23.9

WORKSHOP

背景情况-库研发团队及里程碑

人员	角色	职责描述
王文斌	负责人 需求分析	制定项目计划，包括时间表、预算和资源分配 协调组织相关开发培训、各类会议 收集和理解项目需求、分析业务需求、功能验证
许飞锋	架构设计	技术选型、模块划分、接口定义、评估现有技术的适用性 对技术人员进行开发培训指导 仓颉语言相关特性研究、核心代码编写
龚敏敏 李白全 贾梦雷 代子涵	开发	代码编写 编写自动化测试用例 确保代码的可读性和可维护性 代码优化及性能调优 编写相关文档
齐欣	测试	确保系统功能的正确性和完整性 跟踪缺陷的修复情况 性能对比测试

2023 11

- 项目组成立
- 仓颉语言学习
- 参与华为培训
- Java版本组件研究

2023 12

- 架构设计
- 核心代码编写
- 仓颉语言特性研究
- 团队内部分享

2024 6

- 代码开发完成
- 单元测试
- 性能对比测试
- 内部多次沟通、分享
- 0.0.1版本发布

WORKSHOP

背景情况-库版本发布



- 2024年3月15号
- 版本：v0.0.1

- 所有对外接口
- 支持生命周期管理,并行文件系统等特性
- 分段上传 + 服务端加密

- 华为社区
- 地址:https://gitee.com/HW-PLLab/obs-client-cj

- 用户反馈、优化代码、技术更新

WORKSHOP

库介绍-功能介绍

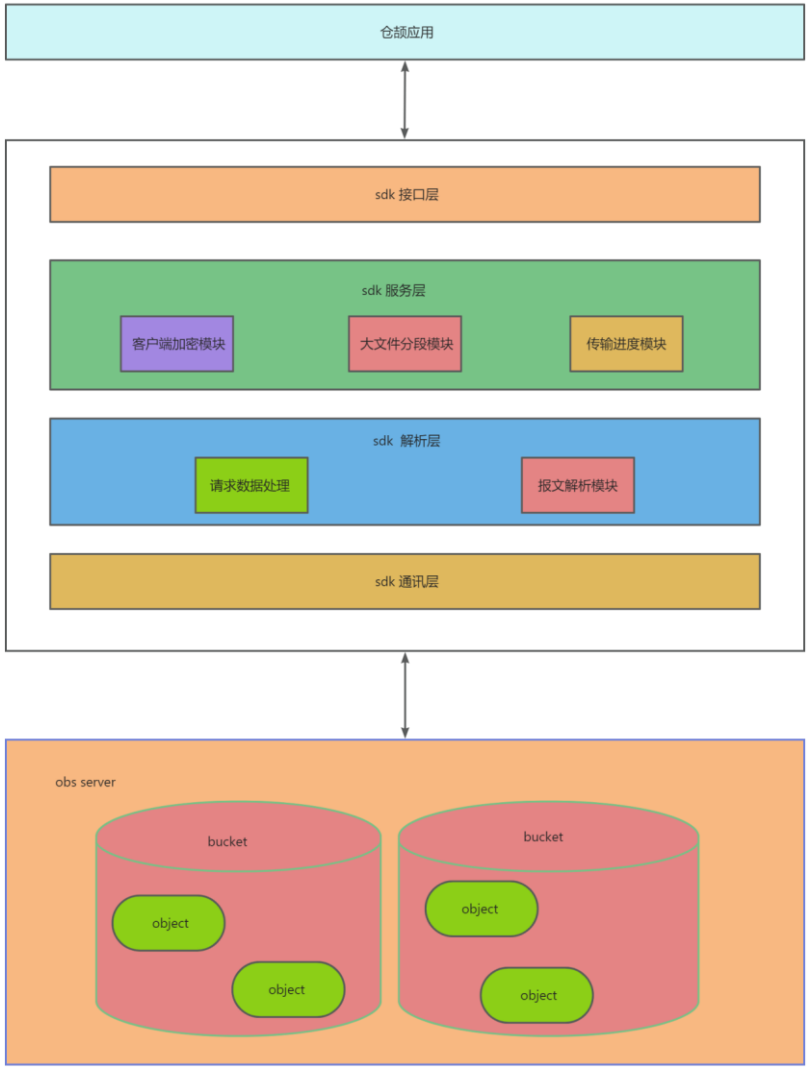
库核心功能和特性：



注：完整功能或接口清单参见社区开源文档

WORKSHOP

库介绍-架构介绍


仓颉应用
sdk 接口层
sdk 服务层
客户端加密模块
大文件分段模块
传输进度模块
sdk 解析层
请求数据处理
报文解析模块
sdk 通讯层
obs server
bucket
object
object
bucket
object
object

库介绍-断点续传

- 创建上传任务和检查checkpoint文件
- 上传段和记录段上传状态
- 合并段,更新checkpoint

优势

- ✓ 加快下载速度,提高吞吐量
- ✓ 在网络问题中快速恢复
- ✓ 重下时节省资源提高效率

```
var uploadCheckPoint: UploadCheckPoint = UploadCheckPoint()
if (uploadFileRequest.isEnableCheckpoint()) {
    prepareWithCheckpoint(uploadFileRequest, uploadCheckPoint)
} else {
    prepare(uploadFileRequest, uploadCheckPoint)
}

// 开始上传
var partResults: ArrayList<PartResult> = uploadfile(uploadFileRequest, uploadCheckPoint)

// 有错误抛出异常,无异常则合并多段
for (partResult in partResults) {
    if (partResult.getIsFailed() && partResult.getException().isSome()) {
        // 未开启，取消多段
        if (!uploadFileRequest.isEnableCheckpoint()) {
            this.abortMultipartUploadSilent(uploadCheckPoint.uploadID.getOrThrow(), uploadFileRequest)
        } else if (uploadCheckPoint.isAbort) {
            this.abortMultipartUploadSilent(uploadCheckPoint.uploadID.getOrThrow(), uploadFileRequest)
            if (uploadCheckPoint.isDeleteUploadRecordFile) {
                ServiceUtils.deleteFileIgnoreException(uploadFileRequest.getCheckpointFile())
            }
        }
        throw partResult.getException().getOrThrow()
    }
}



// 合并多段
var completeMultipartUploadRequest: CompleteMultipartUploadRequest = CompleteMultipartUploadRequest(
    uploadFileRequest.getBucketName(), uploadFileRequest.getObjectKey(), uploadCheckPoint.uploadID.getOrThrow(),
    uploadCheckPoint.partEtags.getOrThrow())
completeMultipartUploadRequest.setRequesterPays(uploadFileRequest.getIsRequesterPays())
completeMultipartUploadRequest.setEncodingType(uploadFileRequest.getEncodingType())
completeMultipartUploadRequest.setIsIgnorePort(uploadFileRequest.getIsIgnorePort())
completeMultipartUploadRequest.setUserHeaders(uploadFileRequest.getUserHeaders())
```

WORKSHOP

库介绍-客户端加密

- 对象数据加密
- 元数据存储加密信息

优势

- ✓ 数据安全
- ✓ 自动解密

```
//上传加密
public func putObjectImplCtr(request: PutObjectRequest): ObsFSFile {
    if (request.getMetadata().isNone()) {
        request.setMetadata(ObjectMetadata())
    }

    if (this.ctrCipherGenerator.isSome()) {
        // 获取AES密钥和初始值
        var object_CryptoIvBytes = getOrGenerateCryptoIvBytes()
        var object_CryptoKeyBytes = getOrGenerateCryptoKeyBytes()
        var buffer: Array<Byte> = Array<Byte>()
        buffer = request.getFile().getOrThrow().readToEnd()
        // 设置加密数据流
        var byteAES = ctrCipherGenerator.getOrThrow().aesEncodePadding(buffer, object_CryptoKeyBytes.getOrThrow(),
            object_CryptoIvBytes.getOrThrow())

        var byteStream = ByteArrayStream()
        byteStream.write(byteAES)
        request.setInput(byteStream)
        var objectMetadata = request.getMetadata()
        // 设置加密信息的自定义头域
        objectMetadata.getOrThrow().addUserMetadata(CTRCipherGenerator.ENCRYPTED_START_META_NAME,
            toBase64String(object_CryptoIvBytes.getOrThrow()))
        if (ctrCipherGenerator.getOrThrow().getMasterKeyInfo().isSome()) {
            objectMetadata.getOrThrow().addUserMetadata(CTRCipherGenerator.MASTER_KEY_INFO_META_NAME,
                ctrCipherGenerator.getOrThrow().getMasterKeyInfo().getOrThrow())
        }
        // 附件加密算法元数据信息
        objectMetadata.getOrThrow().addUserMetadata(CTRCipherGenerator.ENCRYPTED_ALGORITHM_META_NAME,
            CTRCipherGenerator.ENCRYPTED_ALGORITHM)
    }
```

WORKSHOP

库介绍-临时授权访问

- 携带鉴权信息
- 授权其他用户

优势

- ✓ 限制访客用户的访问时长
- ✓ 临时访问

```
//添加失效时间
var expire = request.getExpires()
expire += DateTime.now().toUnixTimeStamp().toSeconds()
url += "&Expires=" + expire.toString()

//添加请求参数
url = ObsUtil.addRequestParametersToUrlPath(url, request.getQueryParams().getOrThrow(), false)
var headers: HashMap<String, String> = HashMap<String, String>()
headers.putAll(request.getHeaders().getOrThrow())
headers.put(CommonHeaders.HOST, hostName)


var urlArray = url.split("?")
var queryString = ""
if (urlArray.size > 1) {
    queryString = urlA
}
var fullUrl = "/" + re
var signUrl = ObsUtil.getfullUrl(fullUrl, hostName, request.getBucketName(), queryString)
//签名

var method = HttpMethodEnum.getName(request.getMethod().getOrThrow()).toAsciiUpper()
var shaStr = AbstractAuthentication.makeServiceCanonicalStringBysignture(headers, method, signUrl, expire)
var signature = ObsUtil.getCryptoString(shaStr, sk.getOrThrow())
signature = signature.replace("\\+", "%2B")
signature = signature.replace("=", "%3D")
signature = signature.replace("/", "%2F")
signature = signature.replace("\\/", "%2F")
url += "&Signature=" + signature
var temporarySignatureResponse = TemporarySignatureResponse(url)
temporarySignatureResponse.getActualSignedRequestHeaders().getOrThrow().putAll(headers)

return temporarySignatureResponse
```

```
Declared in: obs4cj/util/ObsUtil.cj

// In class ObsUtil
(function) public static func getfullUrl(fullUrl: String, hostname: String, bucketName: String, queryString: String): String
```

WORKSHOP

特性分享-函数是一等公民

```
main(): Int64 {
    func f(x: Int) {
        return x
    }
    let a = f
    let square = {x: Int => x * x} // lambda 表达式

    // 函数嵌套定义，以及函数作为返回值
    func g(x: Int) {
        func h() {
            return f(square(x))
        }
        return h
    }

    func h(f: () -> Unit) {
        f()
    }
```

- ✓ 代码更加模块化、灵活和易于维护
- ✓ 函数链式调用和高阶函数

WORKSHOP

特性分享-高效并发

```
let mtx: ReentrantMutex = ReentrantMutex()

for (task in unfinishedTasks) {
    let fut: Future<PartResultDown> = spawn {
        =>
        var partResult = task.call(mtx)
        return partResult
    }
    futures.append(fut)
}

for (future in futures) {
    var tr: PartResultDown = future.get()
    taskResults.append(tr)
}

downloadResult.setPartResults(taskResults)

return downloadResult
```

- ✓ 轻量化用户态线程
- ✓ 减少上下文切换成本

WORKSHOP

特性分享-类库继承扩展

```
extend String {
    public func toBool(): Bool {
        return Bool.parse(this.toAsciiLower())
    }

    public func toInt64(): Int64 {
        return Int64.parse(this)
    }

    public func toInt32(): Int32 {
        return Int32.parse(this)
    }

    public func toDateRfc822(): DateTime {
        return DateUtils.toDateRfc822(this)
    }

    public func toDateIso8601(): DateTime {
        return DateUtils.toDateIso8601(this)
    }
}

extend DateTime {
    public func toStringRfc822(): String {
        return DateUtils.toStringRfc822(this)
    }

    public func toStringIso8601(): String {
        return DateUtils.toStringIso8601(this)
    }
}
```

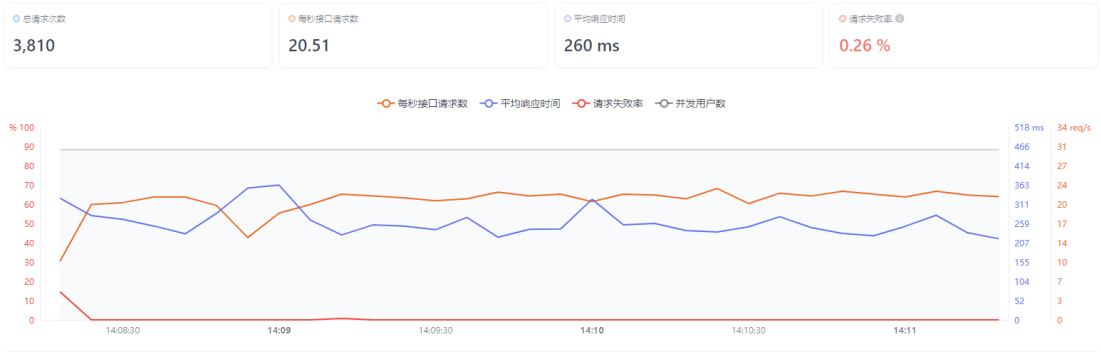
- ✓ 扩展基础库
- ✓ 满足个性化需求

WORKSHOP

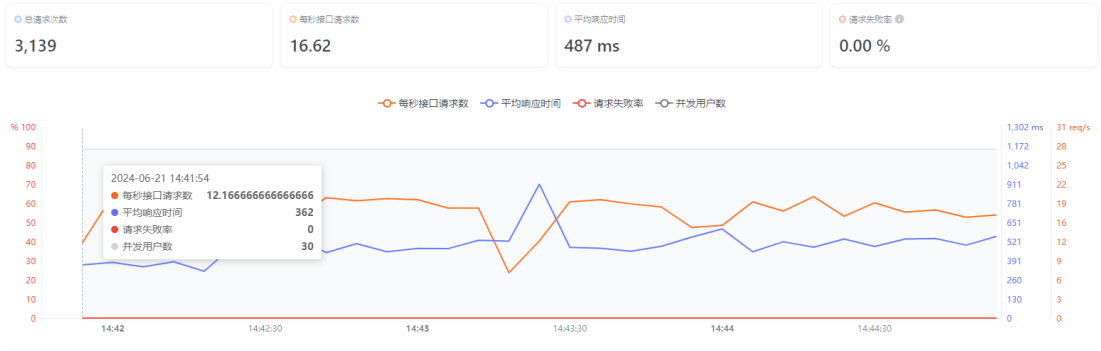
JAVA和仓颉性能对比

- 桶创建
- 文件下载

同等测试环境下，请求数及响应时间都优于Java版



仓颉



Java

WORKSHOP

未来展望

- 代码性能优化
- 多版本兼容
- 集成优秀生态组件

WORKSHOP

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

